

NOTE : Ce chapitre utilise le paris sportif comme terrain d'entraînement pour comprendre les probabilités, les cotes, et la modélisation statistique – des compétences directement transférables à l'analyse de ventes, risques, et prévisions en entreprise.

6.1 LES COTES COMME PROBABILITÉS IMPLICITES

Une cote décimale de 2.50 signifie que le bookmaker évalue la probabilité de cet événement à :

$$P_{\text{implicite}} = 1 / \text{cote} = 1 / 2.50 = 40\%$$

La somme des probabilités implicites de tous les résultats d'un match (1, X, 2) dépasse toujours 100%. Cette différence est la marge du bookmaker, appelée "overround" ou "vig".

EXEMPLE – Match Congo-Brazza vs Léopards DRC :

Cote Congo : 2.10 → P = 47.6%

Cote Nul : 3.40 → P = 29.4%

Cote Léopards : 3.60 → P = 27.8%

TOTAL : P = 104.8% ← 4.8% de marge bookmaker

CODE PYTHON :

```
def analyser_cotes(cote_1, cote_x, cote_2):
    p1 = 1 / cote_1
    px = 1 / cote_x
    p2 = 1 / cote_2
    overround = (p1 + px + p2 - 1) * 100

    # Probabilités "vraies" après suppression de la marge
    total = p1 + px + p2
    return {
        'prob_1': round(p1 / total * 100, 1),
        'prob_x': round(px / total * 100, 1),
        'prob_2': round(p2 / total * 100, 1),
        'marge_bm_pct': round(overround, 2),
        'valeur_seuil': {
            '1': round(1 / (p1 / total), 2),
            'x': round(1 / (px / total), 2),
            '2': round(1 / (p2 / total), 2),
        }
    }
```

VALEUR POSITIVE : Si votre estimation de $P > P_{\text{implicite}}$ du bookmaker, vous avez un pari à "valeur positive" (Expected Value > 0).

6.2 MODÈLE POISSON – PRÉDIRE LES BUTS MARQUÉS

Le modèle de Poisson est la base de la prédiction des buts au football. Il suppose que les buts suivent une distribution de Poisson de paramètre λ (lambda = buts moyens attendus).

```
from scipy.stats import poisson
import numpy as np

def probabilite_score(lambda_dom, lambda_ext, score_max=5):
    """
    lambda_dom : buts attendus par l'équipe domicile
    lambda_ext : buts attendus par l'équipe extérieure
    Retourne une matrice de probabilités de scores
    """
    prob_dom = [poisson.pmf(i, lambda_dom) for i in range(score_max+1)]
    prob_ext = [poisson.pmf(j, lambda_ext) for j in range(score_max+1)]
```

```

# Matrice de scores
matrice = np.outer(prob_dom, prob_ext)

# Probabilités des issues
p_dom = np.tril(matrice, -1).sum() # Domicile gagne (buts dom > ext)
p_nul = np.diag(matrice).sum()     # Match nul
p_ext = np.triu(matrice, 1).sum()   # Extérieur gagne

return {
    'P(1)': round(p_dom * 100, 1),
    'P(X)': round(p_nul * 100, 1),
    'P(2)': round(p_ext * 100, 1),
    'matrice': matrice,
}

# Exemple d'utilisation
resultats = probabilite_score(lambda_dom=1.6, lambda_ext=1.1)
print(resultats)
# → {'P(1)': 47.3, 'P(X)': 28.1, 'P(2)': 24.6}

```

6.3 CALCULER LES λ À PARTIR DES DONNÉES HISTORIQUES

Pour calculer λ , vous avez besoin de l'historique des matchs.
Stocker ces données dans une capsule Gykhamine :

```

class Match(models.Model):
    equipe_dom = models.CharField(max_length=100)
    equipe_ext = models.CharField(max_length=100)
    buts_dom   = models.IntegerField()
    buts_ext   = models.IntegerField()
    date_match = models.DateField()
    competition = models.CharField(max_length=100)

# Calcul des  $\lambda$  moyens sur les N derniers matchs
def calculer_lambda(equipe, role='dom', n_matches=10):
    from django.db.models import Avg
    if role == 'dom':
        qs = Match.objects.filter(equipe_dom=equipe).order_by('-date_match')[:n_matches]
        return qs.aggregate(Avg('buts_dom'))['buts_dom__avg'] or 1.2
    else:
        qs = Match.objects.filter(equipe_ext=equipe).order_by('-date_match')[:n_matches]
        return qs.aggregate(Avg('buts_ext'))['buts_ext__avg'] or 1.0

```

6.4 KELLY CRITERION – QUELLE MISE POUR MAXIMISER LA CROISSANCE ?

La formule de Kelly détermine la fraction optimale de votre bankroll
à parier quand vous avez un avantage statistique.

```

def kelly_mise(prob_estimee, cote_bookmaker, bankroll, fraction=0.25):
    """
    prob_estimee : votre estimation de la probabilité de gain (0-1)
    cote_bookmaker : la cote proposée (ex: 2.50)
    bankroll      : capital disponible en FCFA
    fraction      : sécurité (0.25 = Kelly/4, recommandé)
    """
    b = cote_bookmaker - 1 # Gain net pour 1 FCFA misé
    q = 1 - prob_estimee   # Probabilité de perte

    kelly_fraction = (b * prob_estimee - q) / b

    if kelly_fraction <= 0:
        return {'mise': 0, 'conseil': 'Pari sans valeur – ne pas miser'}

    mise_recommandee = bankroll * kelly_fraction * fraction

    return {
        'kelly_fraction_pct': round(kelly_fraction * 100, 2),
        'mise_recommandee': round(mise_recommandee),
        'conseil': f"Miser {round(mise_recommandee)} FCFA"
    }

```

```
# Exemple
r = kelly_mise(prob_estimee=0.55, cote_bookmaker=2.10, bankroll=50000)
# → mise_recommandee ≈ 1 375 FCFA (prudent, sécurisé)
```

6.5 TRANSFERT VERS LE MONDE DE L'ENTREPRISE

LES MÊMES OUTILS, APPLIQUÉS DIFFÉREMMENT :

CONCEPT BET	ÉQUIVALENT ERP	UTILITÉ PRATIQUE
Probabilité cote	Prob. de défaut	Risque client en microfin
Modèle Poisson	Prévision ventes	Nb de ventes/jour prévu
Kelly Criterion	Allocation stock	Quelle quantité commander
Overround bookmaker	Marge commerciale	Marge réelle sur produit
Backtest historique	Test de stratégie	Valider avant de déployer

RÉSUMÉ DU CHAPITRE 6

- ✓ Cote → probabilité implicite → identifier la valeur positive
 - ✓ Modèle Poisson → prédire des événements discrets (buts, ventes/jour)
 - ✓ Kelly Criterion → allouer les ressources sous incertitude
 - ✓ Tous ces modèles stockent leurs données dans une capsule Django
 - ✓ Les mêmes formules gèrent le risque sportif et le risque business
- SUITE : Chapitre 7 – Ontologie et modélisation conceptuelle